CS 577 Deep Learning
Homework 3: Attention and Transformer Forward

Assignment Specification and Grading Rubric Fall 2026

Release:	Wednesday, September 30, 2026
Due:	Sunday, November 8, 2026, 11:59 p.m. Central Time
Weight:	8% of the course grade
Expected effort:	8–12 focused hours
Work mode:	Individual; identical core assignment for in-person and online students

What You Will Build

Follow the same three-token example through scores, masks, weighted values, multiple heads, and one Transformer block. Compute a small case by hand, implement the forward operations in NumPy, and check them against independent PyTorch operations and controlled input changes. The October 5 lecture supports the multi-head and Transformer portions. No model training, attention backward derivation, or GPU is required.

By the end, you should be able to trace tensor axes, implement stable masked attention, explain what separate heads change, and verify position and causal visibility in a complete forward pass.

Files to Submit

Submit `hw3_<hawk_username>.zip` containing exactly:

1. `hw3_<hawk_username>.pdf`, compiled from the written LaTeX template;
2. `hw3_<hawk_username>.ipynb`, completed with visible outputs and plots; and
3. `hw3_attention.py`, your implementation.

Restart the notebook kernel and run all cells on CPU. Before submitting, run:

```
python -m unittest -v test_hw3_public.py
```

Passing public tests alone does not earn full credit: additional inputs, calculations, reference checks, plots, and explanations are also graded. The usual syllabus late-day policy applies.

Collaboration and Assistance

You may discuss concepts and general debugging strategies, but write, implement, and test your own work. Do not share code, numerical answers, completed equations, or solution-producing prompts. Course materials and official Python, NumPy, PyTorch, Matplotlib, and LaTeX documentation are allowed. Generative AI may help with concepts, syntax, documentation search, or debugging suggestions; it may not generate the core implementation, calculations, experimental interpretation, or written answers submitted for credit. Disclose each tool/person, its purpose, and what you verified or changed. If none was used, state: "No external assistance or generative-AI tools were used."

Notation and the Attention Operation

Use NumPy `float64` throughout. Token positions and feature indices start at zero. The code uses row vectors and batch-first input arrays.

Symbol	Meaning
B	number of independent sequences in the batch
T_q, T_k	query token count and key/value token count
T	shared token count in self-attention, $T_q = T_k = T$
D	features per token before/after multi-head attention
H	number of attention heads
d_k, d_v	query/key features and value features per head
Q, K, V	arrays of query, key, and value vectors
S, A, Y	scaled scores, attention weights, and weighted value outputs

Let i index a query position, j a key position, and r the key positions in the normalization sum. Each score compares one query with one key. Softmax normalizes over *key positions*, separately for every batch, head, and query:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad A_{ij} = \frac{\exp(S_{ij})}{\sum_{r=0}^{T_k-1} \exp(S_{ir})}, \qquad Y = AV.$$

In batched code, transpose only the last two axes of the keys. The output at one query has d_v numbers, regardless of how many keys it reads.

Part 0: Setup and Axis Conventions (5 points)

0.1 Environment and public tests (2 points)

Report Python, NumPy, PyTorch, and Matplotlib versions, the CPU device, seed 577, and the final public-test output in the notebook.

0.2 Name the axes (3 points)

For $Q \in \mathbb{R}^{B\times H\times T_q\times d_k}$, $K \in \mathbb{R}^{B\times H\times T_k\times d_k}$, and $V \in \mathbb{R}^{B\times H\times T_k\times d_v}$, give the shapes of S, A, Y. Explain why normalization is on the last score axis. Explain why a key-validity array of shape (B, T_k) must be reshaped to $(B, 1, 1, T_k)$ before broadcasting.

Part 1: One Attention Calculation (20 points)

Use one batch and one head. Rows are *The*, *cat*, *sleeps*, in order:

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}, \qquad d_k = d_v = 2.$$

These illustrative vectors are supplied, not learned in this exercise.

1.1 All three words are visible (8 points)

Show $QK^\top$, the scaled score matrix, all attention weights, and all output vectors. Show the softmax calculation for the *cat* row and expand its weighted sum of the three value vectors. Give every matrix shape. Round final written values to four decimals; keep full precision in code.

1.2 Only the causal prefix is visible (8 points)

Use $j \le i$: *The* reads *The*; *cat* reads *The, cat*; *sleeps* reads all three. Show the mask, the new weight matrix, and the new outputs. Expand the *cat* output. Explain why the last row is unchanged and why each output still contains two numbers.

1.3 Why divide by the square root (4 points)

For one query vector q and one key vector k, assume all their coordinates are mutually independent, zero-mean, and unit-variance. Show the variance of $\sum_{r=0}^{d_k-1} q_r k_r$ and its scaled version. Explain how extreme scores can concentrate softmax weights. Explain why subtracting a row maximum improves numerical stability without changing softmax.

Part 2: Stable Masked Attention in NumPy (25 points)

Complete `masked_softmax`, `causal_mask`, and `scaled_dot_product_attention`. Use vectorized array operations; no loops over batches, heads, or token positions. Do not mutate inputs or parameters. Use the provided interfaces exactly. Core functions may not call PyTorch, framework attention, or an external Transformer implementation.

2.1 Stable masked softmax (10 points)

In this assignment **Boolean True means allowed**. Apply the mask before normalization. For a row with at least one allowed key, if m_i is the largest *allowed* score in that row:

$$A_{ij} = \begin{cases} \dfrac{e^{S_{ij}-m_i}}{\sum_{r \text{ allowed}} e^{S_{ir}-m_i}}, & j \text{ allowed}, \\ 0, & j \text{ blocked}. \end{cases}$$

Return exact zero for blocked entries; each nonempty row must sum to one. Test finite scores such as $[1000, 999, -1000]$ without overflow. Reject non-Boolean masks and masks that cannot broadcast *to* the score shape.

Fully blocked rows: return all zeros without warnings or NaNs. This is an explicit safe guard for padding, because ordinary softmax of an entirely blocked row is undefined. Such a row has no attention output; exclude padded query rows from downstream losses. This rule does not require an entire Transformer block to output zeros, since it also has residual connections.

2.2 Scaled dot-product attention (10 points)

The input and output contract is:

$$Q : (B, H, T_q, d_k), \quad K : (B, H, T_k, d_k),$$
$$V : (B, H, T_k, d_v), \quad Y : (B, H, T_q, d_v).$$

Return `(Y, cache)`. The cache must contain `scores` (scaled but *before* masking) and `weights`; both have shape (B, H, T_q, T_k). Reject incompatible dimensions with `ValueError`. Support rectangular attention and unequal query/key versus value widths. Reproduce Part 1 using the supplied `hand_case()` and report numerical agreement.

2.3 Padding and causal masks together (5 points)

The notebook supplies two sequences of length four with actual lengths two and four. Construct a key-validity mask of shape $(B, 1, 1, T)$ and combine it with a causal mask of shape $(1, 1, T, T)$ using Boolean AND. Report the combined mask for both examples, and verify zero probability on padded keys and future keys. Then multiply padded value vectors by a large factor and show that valid-query outputs do not change. Demonstrate the fully blocked-row guard separately.

Mask Compatibility with PyTorch

Our `allowed` mask uses True for permitted entries. The `nn.MultiheadAttention` API uses Boolean True for blocked entries; invert our mask. Its key-padding mask uses True for padding. The functional scaled-dot-product attention API uses True for allowed entries. State your reference API/convention. For `nn.MultiheadAttention`, compare nonempty rows.

Part 3: Multi-Head Attention (25 points)

Use the supplied token embeddings `token_case()` for *The/cat/sleeps*:

$$X = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \qquad B = 1, \quad T = 3, \quad D = 4, \quad H = 2, \quad d_h = D/H = 2.$$

Use `initialize_mha(4, seed=577)`. Its four dense learned projection matrices have no biases:

$$W_Q, W_K, W_V, W_O \in \mathbb{R}^{D \times D}.$$

They are fixed sample parameters, not a trained language model.

For self-attention, project the complete input before splitting features:

$$Q_{\text{full}} = XW_Q, \quad K_{\text{full}} = XW_K, \quad V_{\text{full}} = XW_V.$$

Reshape (B, T, D) to (B, T, H, d_h) and transpose to (B, H, T, d_h). Each head computes its own scores and weights, using $\sqrt{d_h}$ as the divisor. After attention, merge in head order and apply W_O:

$$Y_{\text{MHA}} = \text{Concat}(Y_0, \ldots, Y_{H-1})W_O.$$

$Y_0, \ldots, Y_{H-1}$ are the weighted value outputs of heads 0 through $H-1$. For cross-attention, queries come from X_{query} and keys/values from X_{context}. Their token counts may differ; their batch and feature widths must match.

3.1 Implement and trace the heads (12 points)

Complete `split_heads`, `merge_heads`, and `multi_head_attention`. Reuse Part 2. Report shapes from projection to final output, the two 3×3 weight matrices, and the final 3×4 output. Plot both attention maps with query words on rows, key words on columns, and a shared color scale from zero to one. The docstrings specify all cache keys. Explain why slicing unprojected input features cannot replace the learned full-width projections.

3.2 Independent PyTorch reference (6 points)

Create `nn.MultiheadAttention` with `embed_dim=4`, `num_heads=2`, `bias=False`, `dropout=0.0`, and `batch_first=True`, on CPU in `float64`. Copy the NumPy parameters: PyTorch linear weights store the transpose of our matrices. Its packed input weight vertically concatenates $W_Q^\top, W_K^\top, W_V^\top$; its output weight is $W_O^\top$. Request `average_attn_weights=False`. Compare both output and per-head weights for (a) unmasked self-attention, (b) causal self-attention, and (c) rectangular cross-attention with two query tokens and three context tokens. Report maximum absolute errors. Expected agreement on these supplied cases is within 10^{-9}.

3.3 What changes when we change the heads (7 points)

Keep X and all four projection matrices fixed. Compare $H = 1$ with $H = 2$: report per-head feature widths, score shapes, MHA parameter counts, and maximum output difference. Then set one head's *output* to zero before merging and applying W_O, once for each head. Report both changes from the complete two-head output. Explain why fixed total width keeps the projection parameter count unchanged, why the outputs can differ, and why untrained attention maps do not establish that a head learned a particular semantic role.

Part 4: One Transformer Block and Controlled Checks (20 points)

Use `initialize_block(model_dim=4, ff_dim=8, seed=577)`. Complete `transformer_block` using the *provided* feature-wise `layer_norm` helper. This exercise uses pre-normalization, ReLU, and no dropout. The feed-forward network acts separately on each token. Define:

$$\begin{aligned} N_1 &= \mathrm{LN}_1(X), \\ U &= X + \mathrm{MHA}(N_1), \\ N_2 &= \mathrm{LN}_2(U), \\ F &= \mathrm{ReLU}(N_2 W_1 + b_1) W_2 + b_2, \\ Z &= U + F. \end{aligned}$$

Here $\mathrm{MHA}(N_1)$ forms queries, keys, and values from N_1 using the learned projections. LN_1 and LN_2 are the two LayerNorm operations. $W_1 : (D, D_{\mathrm{ff}})$, $b_1 : (D_{\mathrm{ff}})$, $W_2 : (D_{\mathrm{ff}}, D)$, and $b_2 : (D)$; $D_{\mathrm{ff}} = 8$ is the hidden feature width of the feed-forward network. Every N_1, U, N_2, F, Z has shape (B, T, D). Each normalization has learned feature scale and offset vectors of shape (D); the helper uses population variance and $\epsilon = 10^{-5}$.

4.1 Implement and independently verify the block (8 points)

Return `(Z, cache)` with the keys in the docstring. Report all intermediate shapes. Independently recreate this same block using PyTorch normalization, your PyTorch MHA reference, and PyTorch affine/ReLU operations. Compare final outputs on the unmasked fixed token case within 10^{-9}. Show that setting W_O, W_2, b_2 to zero makes the block return X through the residual paths.

4.2 Add position and reorder the same tokens (6 points)

Complete `sinusoidal_positions`. For even width D, let p be a token position and $m \in \{0, \ldots, D/2 - 1\}$ index two adjacent feature columns:

$$\omega_m = 10000^{-2m/D}, \qquad \mathrm{PE}_{p,2m} = \sin(p\omega_m), \qquad \mathrm{PE}_{p,2m+1} = \cos(p\omega_m).$$

Thus for $D = 4$, columns 0/1 use rate 1, and columns 2/3 use rate 0.01, in *radians per position*. The position matrix has shape (T, D). Show all three four-number position vectors. Add them feature by feature: $X_{\mathrm{pos}} = X + \mathrm{PE}$, broadcasting over the batch axis. Do not concatenate positions to token vectors.

Use the token order *sleeps/The/cat*, corresponding to indices $[2, 0, 1]$. Without position and without a causal mask, compare the block output on reordered tokens with the correspondingly reordered original output. With position, keep the position vectors attached to their *slots*, reorder only token embeddings, and repeat the comparison. Report both maximum differences and explain what each comparison demonstrates.

4.3 Can a later input affect an earlier output (6 points)

Use $X + \mathrm{PE}$. Add $[5, -3, 2, -4]$ to the embedding at the last token only. Compute the maximum output change over the *first two* positions (a) with the causal mask and (b) without it. Interpret the results through attention, token-wise normalization/FFN, and residuals. Causal prefix differences should be below 10^{-9}. The supplied unmasked case should give a detectable change. Explain why the diagonal is permitted when position i predicts token $i + 1$.

Part 5: Architecture Comparison and Submission Quality (5 points)

5.1 Trace one generation step (3 points)

Compare translating *The cat sleeps* to *Le chat dort*:

1. In an encoder–decoder Transformer, begin with the supplied BOS (beginning of sequence) token. Identify the query vector and the source key/value vectors used by cross-attention when predicting *Le*. Then explain the next step: the input position *Le* predicts *chat*. Which target inputs and source inputs are visible? Is BOS supplied or predicted in this example?
2. In a decoder-only Transformer with the prompt *French : The cat sleeps =*, identify the position whose final vector predicts *Le*. During causal prefill, can the *The* position read the later *cat* and *sleeps* positions? Distinguish all prompt tokens being available to the program from each query's permitted attention keys.

No code or numerical translation-quality evaluation is required for this part.

5.2 Reproducibility and presentation (2 points)

Use meaningful names, correct filenames, visible outputs, labeled attention maps, a readable PDF with mathematical notation, and the assistance/AI disclosure. Include the exact public-test command and your actual results. Do not invent runs or claim that random sample parameters learned translation.

Grading Rubric

Part	Points	Full-credit evidence
0. Setup and axes	5	Versions/tests; correct axis meanings and mask broadcast
1. Attention calculation	20	Shapes, scores, weights, outputs; causal visibility; scaling/stability reasoning
2. Masked attention	25	Stable vectorized code, mask guard, rectangular shapes, padding invariance
3. Multi-head attention	25	Projection/split/merge trace; per-head maps; three PyTorch checks; controlled head comparisons
4. Transformer block	20	Pre-norm implementation/reference, positions, permutation and causal checks
5. Architecture and quality	5	Correct generation-step comparison; reproducible and readable submission; disclosure
Total	**100**	

Grading Boundaries

Public tests are representative, not exhaustive. Additional tests use the same interfaces and may vary batch sizes, head counts, feature widths, rectangular token counts, masks, or parameter values. They do not require training, backpropagation, caching, FlashAttention, or a GPU. Floating-point results are compared with tolerances: normally `atol=1e-10, rtol=1e-9` for NumPy cases and maximum error below 10^{-9} for the supplied PyTorch/invariance checks. Written hand calculations are graded with four-decimal rounding tolerance.

No credit for a core implementation replaced by a prohibited attention API. Missing reasoning where requested earns at most half credit for that item. Loops where vectorization is required may lose up to half of the affected function's points. Missing files, outputs, or a runnable notebook lose credit within affected items; avoid double penalties for one underlying failure. Missing disclosure loses up to two quality points. Academic-honesty violations are handled under course policy.

References

Vaswani et al., *Attention Is All You Need*, 2017, Sections 3.2 and 3.5: `https://arxiv.org/abs/1706.03762`. PyTorch official documentation for multi-head attention, functional scaled-dot-product attention, and layer normalization: `https://docs.pytorch.org/docs/stable/`. The homework uses the specified pre-norm block order, which differs from the original paper's post-norm block. The assignment's docstrings govern its mask and fully blocked-row conventions.